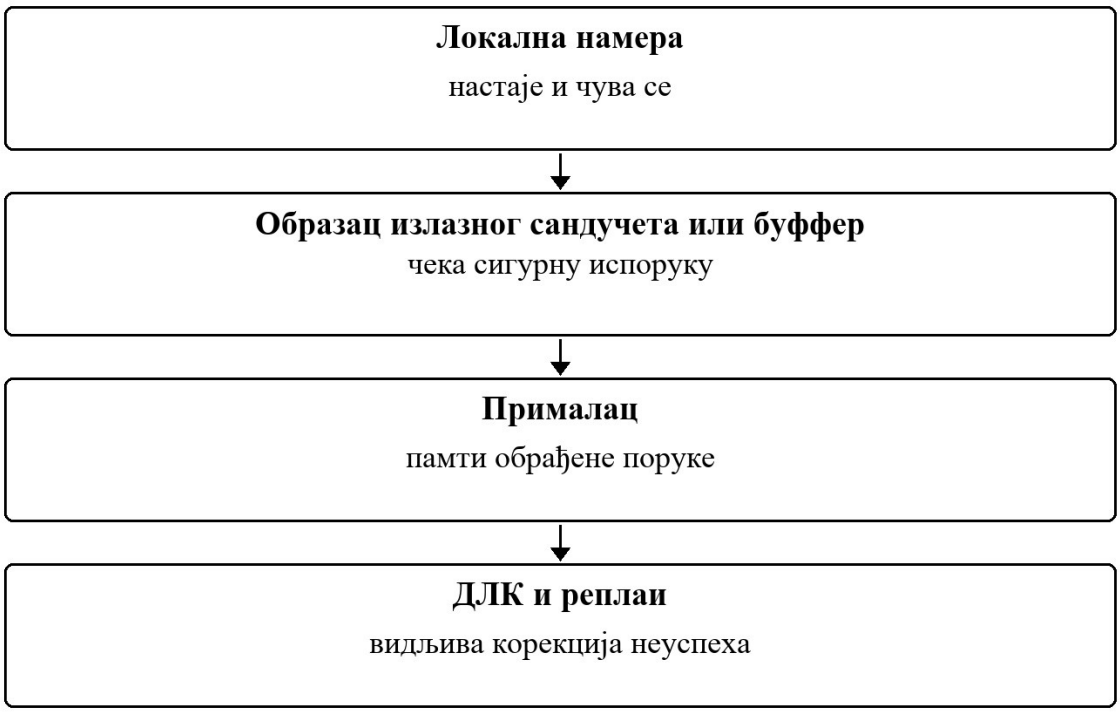
Рад без везе и поуздана испорука

Основе дистрибуираног програмирања · 07

Прекид везе не мора да заустави локални рад, али уводи питање шта се десило док друга страна није била доступна. Буффер, реплаи и образац излазног сандучета обрасци чувају намеру и историју, док дедупликација спречава да поновна испорука створи други пословни ефекат.

Поуздана испорука није обећање да мрежа никада неће погрешити. Она је скуп јасних одговорности који омогућава да се порука сачува, поново пошаље, објасни ако не прође и обради једном са становишта пословног ефекта.



Слика 1. Систем који чува смисао поруке кроз прекид и поновни контакт

Дистрибуирана својства настају у односу између компоненти, времена и граница одговорности.

Фокус вежбе

Материјал објашњава дистрибуирани проблем, последице избора и границе гаранције коју систем може да пружи.

7.1 Испорука и обрада су одвојене одговорности

- Порука која не пролази не нестаје тихо, већ добија видљив разлог.
- Поновљена испорука је очекивана и прималац је препознаје.
- Локални капацитет и политика одбацивања морају бити експлицитни.

Питање за дискусију

Коју поруку бисте морали сачувати током прекида везе, а коју бисте смели изгубити уз јасно обавештење? Зашто?

7.2 Како се ова тема види у развоју

Тим најпре одлучује шта може да се задржи локално, колико дуго и шта се дешава при попуњеном капацитету. Затим проверава рецоннецт сценарије и показује да се порука може пратити од локалне намере до коначног пословног исхода.

Честе грешке које треба препознати

- Буффер без ограничења и без политике при попуњености.
- Реплаи који заобилази исту валидацију као први покушај.
- Дедупликација ослоњена само на податак који се може променити.

Веза са остатком градива

Овај принцип се надовезује на уговоре, отказ сценарије, могућност посматрања система и тестирање. Иста логика важи без обзира на конкретну технологију или домен система.

7.3 Пример: рецоннецт после локалног рада

Удаљена компонента наставља локални рад док нема везу и чува поруке за касније. Када се поново повеже, неке од тих порука су можда већ стигле другим путем. Поуздана испорука зато не значи да се све шаље без провере, већ да се поруке могу идентификовати, поновити и обрађивати без дуплог пословног ефекта.

- Буффер има ограничење и јасно правило када више нема простора.
- Образац излазног сандучета чува намеру заједно са пословном променом коју представља.
- Прималац води евиденцију обрађених порука, а деад-леттер чува неуспехе за контролисан реплаи.

Како се сценарио анализира

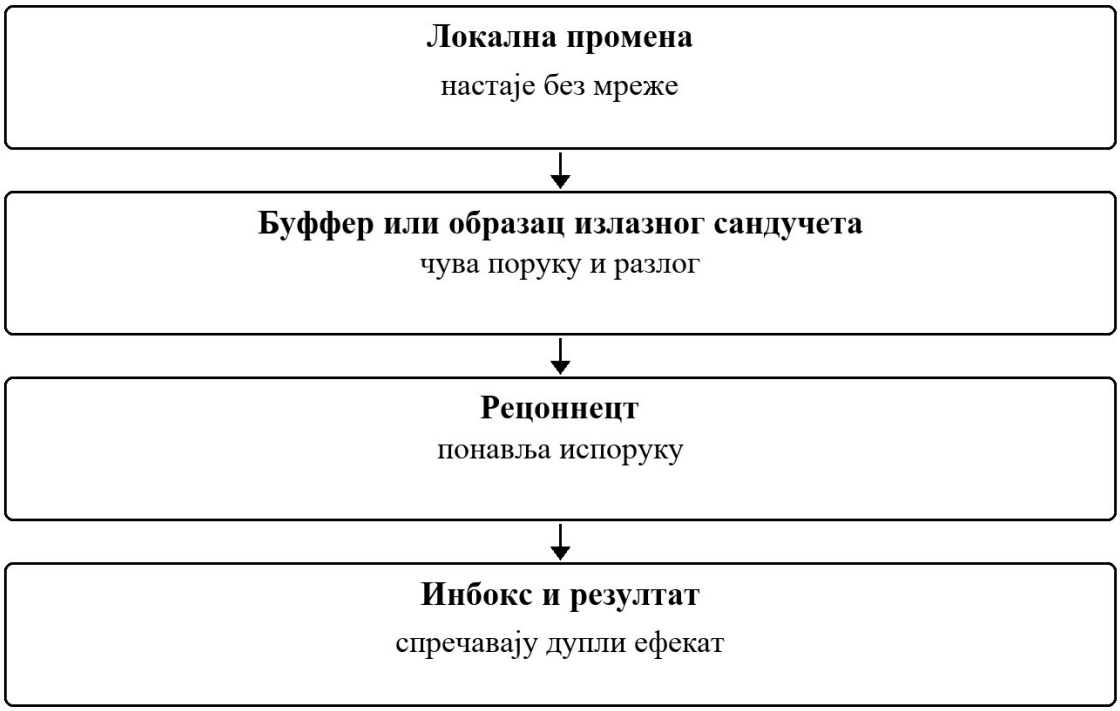
Најпре раздвојите оно што систем зна од онога што претпоставља. Затим одредите стабилан идентитет операције и видљив исход који се може проверити.

7.4 Од принципа до доказа

Дистрибуирани принцип је користан тек када систем има јасан, проверљив одговор на његов сценарио отказа. Добар доказ не показује само да је нормалан ток успео, већ и да кашњење, дупликат, прекид или касна порука не мењају пословно значење операције.

- Именовати стање које је ауторитативно и компоненту која има право да га мења.
- Навести шта се дешава када одговор изостане или стигне након истека рока.
- Понављати контролисани сценарио док се исход може објаснити и независно проверити.

7.5 Шема рада без везе



Слика 2. Шема рада без везе

Прекид везе мења тренутак испоруке, али не сме да избрише намеру нити да је произвољно удвостручи.

Шему користите као кратку проверу: за сваки корак треба умети навести власника одлуке, поруку или стање које се преноси и сценарио отказа који може променити ток.

7.6 Упоредни преглед елемената тока

Табела 1. Елемент и Улоīа у ток̄у

Елемент	Улога у току	Питање које треба поставити
Локална намера	настаје и чува се	Шта се дешава када локална намера није доступан или даје застарелу информацију?
Образац излазног сандучета или буффер	чека сигурну испоруку	Шта се дешава када образац излазног сандучета или буффер није доступан или даје застарелу информацију?
Прималац	памти обрађене поруке	Шта се дешава када прималац није доступан или даје застарелу информацију?
ДЛК и реплаи	видљива корекција неуспеха	Шта се дешава када длк и реплаи није доступан или даје застарелу информацију?

Поређење појмова

Сваки елемент тока решава други део проблема. Стабилан дизајн не пребацује одговорност једног елемента на други само зато што је то краткорочно једноставније.

7.7 Питања за анализу дистрибуираног понашања

Када се анализира дистрибуирани ток, није довољно описати само срећан пут. Потребно је именовати стање које се мења, границу одговорности, поруку или време које може закаснити и исход који систем мора сачувати упркос неизвесности.

1. Порука која не пролази не нестаје тихо, већ добија видљив разлог.

2. Поновљена испорука је очекивана и прималац је препознаје.

3. Локални капацитет и политика одбацивања морају бити експлицитни.

4. Који контролисани сценарио би показао да се пословно значење операције не мења при кашњењу, дупликату или прекиду?

Питање за вежбу

Коју поруку бисте морали сачувати током прекида везе, а коју бисте смели изгубити уз јасно обавештење? Зашто?

7.8 Извршиви пример

Пример чува намеру кроз прекид везе. Станица свако мерење најпре уписује у локални образац излазног сандучета, а слање је одвојен корак који се понавља док не стигне потврда. Пошто потврда може да се изгуби и после успешне испоруке, иста порука понекад стигне два пута; центар је зато примењује само први пут.

Пример 1. Покретање демонстрације и тестова

```
cd examples/odp-dotnet/vezba-07-rad-bez-veze-outbox
dotnet run --project src/Odp.Vezba07.ConsoleUi
dotnet test Odp.Vezba07.sln
```

Правило у коду

Пример 2. examples/odp-dotnet/vezba-07-rad-bez-veze-outbox/src/Odp.Vezba07.Application/Flushing/FlushOutboxHandler.cs

```
public FlushReport Flush()
{
    var pending = ordering.PendingInOrder(outbox.All());
    if (pending.Count == 0)
        return new(0, 0, OutboxCodes.NothingPending);

    var delivered = 0;
    foreach (var message in pending)
    {
        if (!uplink.TrySend(message))
            return new(delivered, pending.Count - delivered,
OutboxCodes.DeliveryUnconfirmed);

        outbox.MarkDelivered(message.Position);
        delivered++;
    }

    return new(delivered, 0, OutboxCodes.Flushed);
}
```

Тест који задаје квар

Пример 3. examples/odp-dotnet/vezba-07-rad-bez-veze-outbox/tests/Odp.Vezba07.Tests/Application/OutboxFlowTests.cs

```
[Test]
public void Flush_WhenAckIsLost_ResendsAndTheCenterAppliesTheMessageOnce()
{
    _record.Record("m-1", "a");
    _record.Record("m-2", "b");
    _uplink.LoseNextAck = true;

    var first = _flush.Flush();
    var second = _flush.Flush();

    using (Assert.EnterMultipleScope())
    {
        Assert.That(first, Is.EqualTo(new FlushReport(0, 2,
OutboxCodes.DeliveryUnconfirmed)));
        Assert.That(second, Is.EqualTo(new FlushReport(2, 0, OutboxCodes.Flushed)));
        Assert.That(_center.AppliedPayloads, Is.EqualTo(new[] { "a", "b" }));
        Assert.That(_center.DuplicatesIgnored, Is.EqualTo(1));
    }
```

```
}
```

Испис демонстрације

Пример 4. дотнет рун --пројецт src/Odp.Vezba07.ConsoleUi

```
Veza je u prekidu.
  merenje m-1 -> message_queued
  merenje m-2 -> message_queued
  merenje m-3 -> message_queued
  pokušaj slanja bez veze -> delivery_unconfirmed, isporučeno 0, preostalo 3
Veza se vratila, ali se prva potvrda gubi.
  prvi pokušaj posle povratka -> delivery_unconfirmed, isporučeno 0, preostalo 3
  drugi pokušaj -> outbox_flushed, isporučeno 3, preostalo 0

Centar je primenio 3 poruke redom:
  signal -91 dBm
  signal -88 dBm
  signal -97 dBm
Ignorisanih duplikata: 1
```

Шта треба уочити

- Упис мерења успева и када везе нема; слање је одвојен корак.
- После повратка везе поруке стижу редом којим су настале.
- Изгубљена потврда изазива поновно слање, а центар поруку примењује једном.

Задатак уз пример

Ограничити величину образац излазног сандучета-а (`Capacity`). Када је пун, ново мерење се одбија кодом `outbox_full` уместо да се тихо изгуби старо. Правило покрити тестом у коме веза остаје у прекиду.

Рад на вежби — примена на пројекат

На додељеној пројектној целини применити принцип из теме „Рад без везе и поуздана испорука". Идентификовати конкретну дистрибуирану одлуку, имплементирати је и припремити поновљив сценарио отказа и тест који тим може да покаже на пројектној контролној тачки.